

Real-world (make it feel alive with streaming) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Explain what a REST API is, in 3 sentences."},
]

stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    stream=True,
)

for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")

print()
```

Expected output: the 3-sentence explanation appearing on screen piece by piece, noticeably faster to *start* than the same call without `stream=True`.

This works and shows the streaming effect clearly. It doesn't keep track of the full reply text anywhere, so if you needed to save it (into a conversation history list, for example) you'd have nothing to append.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `stream_reply()` returns the accumulated text

```
def stream_reply(history: list) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
    return full_reply

def main() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain what a REST API is, in 3 sentences."}
    ]
    reply = stream_reply(history)
    history.append({"role": "assistant", "content": reply})

if __name__ == "__main__":
    main()
```

Expected output: identical streamed text to Basic, plus `history` now correctly ends with the assistant's full reply appended.

Difference from Basic: `stream_reply(history)` returns the full accumulated text once streaming finishes, so it can plug directly into the conversation-memory loop from the previous exercise — call it instead of `get_reply(history)`, append its return value to `history`, and you have a chatbot

with both real memory and streamed output. Full type hints document that it takes a history list and hands back the finished reply as a string, hiding the chunk-by-chunk detail from the caller.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — guarded against an empty- `choices` chunk, and a mid-stream failure

```
import openai

def stream_reply(history: list[dict]) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    try:
        for chunk in stream:
            if not chunk.choices:
                continue # a usage-only chunk carries no delta content
            piece = chunk.choices[0].delta.content
            if piece:
                print(piece, end="")
                full_reply += piece
    except openai.APIError as e:
        print(f"\n[reply cut off - {type(e).__name__}]")

    print()
    return full_reply
```

Expected output on a normal run: identical to Intermediate. **On a mid-stream failure** (simulated, since it's hard to trigger for real on demand): whatever text had already streamed in stays on screen, followed by a clearly labeled `[reply cut off - ...]` marker, and `stream_reply()` still returns the partial text instead of raising all the way up and losing it.

Approach 2 — request token usage on the stream, safely

This is the scenario the `chunk.choices` guard exists for: turning on `stream_options={"include_usage": True}` gets you a token count for a streamed reply (the streaming equivalent

of `response.usage` from the Failure exercise), but only because the final chunk carries no `choices` at all.

```
def stream_reply_with_usage(history: list[dict]) -> tuple[str, int | None]:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
        stream_options={"include_usage": True},
    )

    full_reply = ""
    total_tokens = None
    for chunk in stream:
        if chunk.usage is not None:
            total_tokens = chunk.usage.total_tokens
        if not chunk.choices:
            continue
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece

    print()
    return full_reply, total_tokens
```

Expected output:

Explain what a REST API is, in 3 sentences... (streamed text)

(full_reply, total_tokens) == ("Explain what a REST API is...", 87)

Without the `if not chunk.choices: continue` guard, this function would crash with `IndexError: list index out of range` on the very last chunk — right as the reply was finishing successfully, which is a particularly confusing place for a crash to happen.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `stream_reply()` works correctly as long as every chunk has exactly one item in `choices` — true for a plain streamed call, but not guaranteed in general. Approach 1 adds the `chunk.choices` guard plus a `try/except` around the whole loop, so neither an empty-choices chunk nor a genuine mid-stream failure crashes the function — it returns whatever text it has instead. Approach 2 is the concrete reason the guard matters: turning on `include_usage` for real token counts on a streamed call *always* sends a final chunk with an empty `choices` list, so this function would be broken without the exact same guard Approach 1 added defensively.

Which one should you actually write? Intermediate's version is enough while you're only ever calling with plain `stream=True` and no usage tracking. Reach for Approach 1's guard-and-try/except the moment this code needs to run unattended (which is true of this document's Build Task) — a crash on the last chunk of an otherwise-successful reply is a bad, confusing failure mode to ship. Add Approach 2's `include_usage` once you need real token/cost numbers for streamed replies too, not just non-streamed ones — the same numbers the Failure exercise reads off `response.usage` for a normal call.